EngSci 721

Inverse Problems and Learning From Data

Oliver Maclaren (oliver.maclaren@auckland.ac.nz)

Module overview

Inverse Problems and Learning From Data (*Oliver Maclaren*)
[~14 lectures/3 tutorials]

1. *Basic concepts* [5 lectures + 1 Tutorial]
   Forward vs inverse problems. Well-posed vs ill-posed problems. Algebra and calculus of inverse problems (left and right inverses, generalised and pseudo inverses, resolution operators, matrix calculus). Representing higher dimensional problems (image data etc).
2. *Instability and regularisation in linear and nonlinear problems* [6 lectures + 1 Tutorial]
   Instability and related issues for generalised inverses. Introduction to regularisation and trade-offs. Tikhonov regularisation. Higher-order Tikhonov regularisation. Sparsity and regularisation using different norms. Truncated singular value decomposition. Iterative regularisation, including stochastic/mini-batch gradient descent.
3. *Further topics* [3 lectures + 1 Tutorial]
   Regularisation parameter choice, including statistical and machine learning views of regularisation. Confidence sets for linear and nonlinear models. Physics-informed machine learning and neural networks.

Lecture 12: From Landweber iteration to stochastic gradient methods

Topics:

- Least squares as a finite sum over observations
- Full-batch, mini-batch, and single-row updates
- Batches, epochs, and data passes
  - Fixed and decreasing learning rates
  - Semi-convergence and early stopping

Eng Sci 721: Lecture 12.

Stochastic / minibatch gradient descent:

From Landweber iteration to SGD

Basically:

→ Decomposing least squares objective into partial sums

→ using these partial sums during updates instead of full sums

→ shuffle partial sums = stochastic

Inverse: Landweber → Kaczmarz

ML: batch → minibatch → SGD

Gradient descent in machine learning?

- Gradient descent is also widely used in machine learning (ML), particularly in training (deep) neural networks

- While first-order optimisers (ie based on first derivatives) are not very popular in the pure optimisation community, & improved methods have also been favoured in inverse problems, first order gradient descent methods are surprisingly widely-used & competetive in ML!

  ↳ in particular, a family of variants usually called 'stochastic gradient descent' (SGD) is the 'default'/'go to' approach

  → improvements have been proposed but it's still pretty good!

---

let's look at ingredients!

Recall Landweber iteration:

$$x_{k+1} = x_k + \alpha A^T (y - A x_k)$$

→ typically more memory efficient than eg factorisation methods

However

→ can still be expensive with very large (eg whole internet...) datasets

→ data may be stored in distributed way across multiple physical or virtual locations

→ data may arrive sequentially in time ('online')

Solution: 'per observation' loss

First write i^{th} row of A as a_i^T

$$i \to \begin{bmatrix} \text{—} \; a_i^T \; \text{—} \end{bmatrix} \Big\} A$$

ie $a_i^T = \text{row}_i(A) \quad (= r_i^T(A))$

(recall lecture 4)

& note

$$\begin{bmatrix} \\ \\ \vdots \end{bmatrix} \begin{bmatrix} x \end{bmatrix}$$

$$= \begin{bmatrix} x \\ x \\ \vdots \end{bmatrix}$$

$$= \begin{bmatrix} — \; a_1^T x \; — \\ — \; a_2^T x \; — \\ \vdots \end{bmatrix}$$

→

Now least squares can be decomposed:

('loss' in ML)

$$\min_x l(x) = \min_x \|y - Ax\|^2$$

$$= \min \left\| \begin{bmatrix} y_1 \\ y_2 \\ \vdots \end{bmatrix} - \begin{bmatrix} -a_1^T- \\ -a_2^T- \\ \vdots \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \end{bmatrix} \right\|^2$$

$$= \min \left\| \begin{bmatrix} y_1 \\ y_2 \\ \vdots \end{bmatrix} - \begin{bmatrix} -a_1^T x_1 - \\ -a_2^T x_2 - \\ \vdots \end{bmatrix} \right\|^2$$

$$= \min \left\| \begin{bmatrix} y_1 - a_1^T x \\ y_2 - a_2^T x \\ \vdots \end{bmatrix} \right\|^2$$

$$= \min \; (y_1 - a_1^T x)^2 + (y_2 - a_2^T x)^2 + \dots$$

$$= \min \sum_{i=1}^m (y_i - a_i^T x)^2$$

$$= \min \sum_{i=1}^m l_i(x)$$

("per observation loss")

→ Many other objectives in stat. & ML also have additive decomp.

[Side note: Expected squared error?

$$\min \sum_{i=1}^m (y_i - a_i^T x)^2$$

same soln as

$$\min \frac{1}{m} \sum_{i=1}^m (y_i - a_i^T x)^2$$

→ Can be interpreted as empirical (sample-based) estimate of 'population' expected loss:

$$\mathbb{E}\left[(y - a^T x)^2\right] \approx \frac{1}{m} \sum_{i=1}^m (y_i - a_i^T x)^2$$

$\approx \mathbb{E}$

random vars

sample realisations

→ hence often divide by number of data points & call 'empirical loss' or 'empirical risk'

(decision theory interp.)

From loss/obj. to gradient

In general

$$\nabla_x \frac{1}{m} \sum_{i=1}^{m} l_i(x) = \frac{1}{m} \sum_{i=1}^{m} \nabla l_i(x)$$

(linearity. Rep. version $\nabla \mathbb{E} = \mathbb{E} \nabla$ that requires some regularity cond.)

ie gradient of total/expected/av. loss

=

total/expected/av of gradients of per data point loss

→ in stat terms the empirical version is estimate of population version.

→

Idea: use random subsets of data to estimate gradient & use estimates in iterations.

Eg choose random subset B of b observations & compute estimate g_B of ∇

$$g_B(x) = \frac{1}{b} \sum_{i \in B} \nabla_i l_i(x)$$

Updates of form:

$$x_{k+1} = x_k - \alpha\, g_B(x_k),$$

usually descent form:

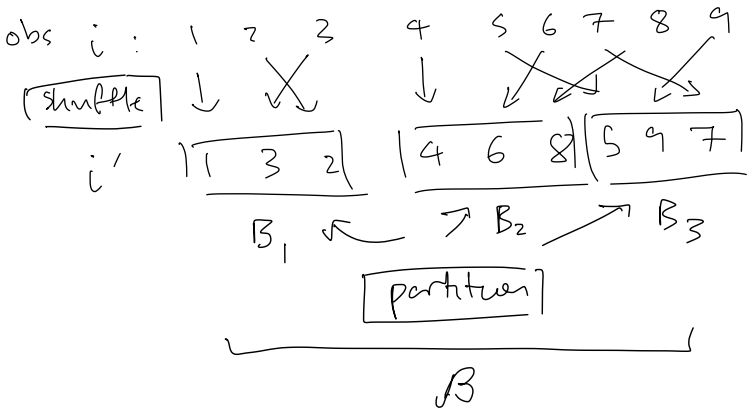
$$x_{k+1} = x_k - \eta_k\, g_B(x_k)$$

η_k: descent; let param. dep. on k.

Call 'learning rate' in ML & use η.

Using in gradient estimates

→ How do we divide & use data?



- shuffle data points
- group into 'mini batches'

↳ whole data set called 'batch'.

Using in gradient estimates:

→ Stochastic & mini batch gradient descent

The term 'stochastic gradient descent' is used for both a general family of methods & a special case of this family

More precisely we should call these

1. Standard or 'batch' gradient descent
2. Stochastic gradient descent
3. 'mini-batch' gradient descent

But we often call 2 & 3 'stochastic gradient descent' for short.

Minibatch gradient descent

- We will randomly divide training data into non-overlapping subsets called minibatches
- loop through minibatches, updating as we go based on minibatch gradient until all minibatches used
- That gives n_mbatch='number minibatches' updates or iterations & one 'epoch' (all minibatches seen) so far.
- we then repeat: redivide training data, iterate through minibatches
- do this for n_epoch = 'number of epochs'
- gives n_mb × n_epoch total parameter updates (iterations)

Terms: iterations (parameter updates)
minibatches (data subsets in partition)
epochs (## times seen whole dataset)

Algorithm: Mini-batch gradient descent for $\frac{1}{m}\|y - f(x)\|^2$ (nonlinear least sq).

$\alpha \leftarrow$ learning rate (fixed here; can vary in general)

$x_0 \leftarrow$ initial parameter-estimate for iteration 1

For i = 1 : n_epoch

  Randomly divide dataset into n_mbatch minibatches

  For j = 1 : n_mbatch

    iteration k = (i-1)·n_mbatch + j

    $x^0_k = x_{k-1}$ intial guess

    Compute $g_k = \nabla \frac{1}{b}\|y_{B_j} - f_{B_j}(x_k)\|^2$ for minibatch B_j of epoch i

    Update $x_k \leftarrow x^0_k - \alpha g_k$

    record loss $f_{ij} = f_k = \frac{1}{b}\|y_{B_j} - f_{B_j}(x_k)\|^2$ for minibatch

  Epoch loss(i) = $\frac{1}{n_{mb}} \sum_j f_{ij}$

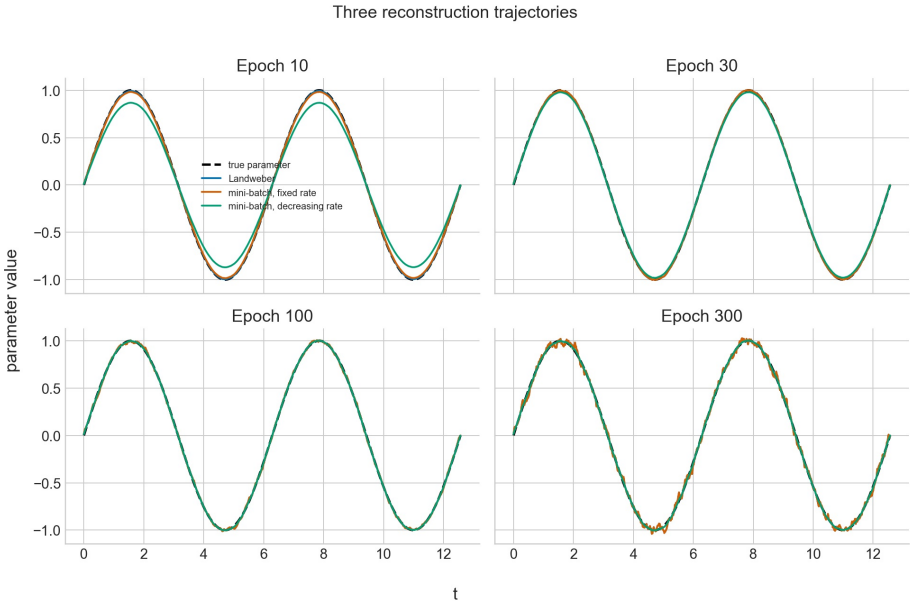
'online' (averaged vs recomputed)

Notes

- don't need to load whole dataset into memory
- often improves 'generalisation' ie regularisation towards good out-of-sample prediction models; learning rate dependent tho.
- can analyse via estimators of expectations but...
- still not well-understood!
- gradients computed for large neural networks via 'fancy chain rule'
  - pytorch & tensorflow etc implement derivative propagation for you!
- Many improvements, tricks etc possible!
  - at minimum, need varying η_k, eg decreasing with k.
- Estimate of average gradient: divide by minibatch size
- Estimate of total gradient: divide by minibatch size & multiply by total data size

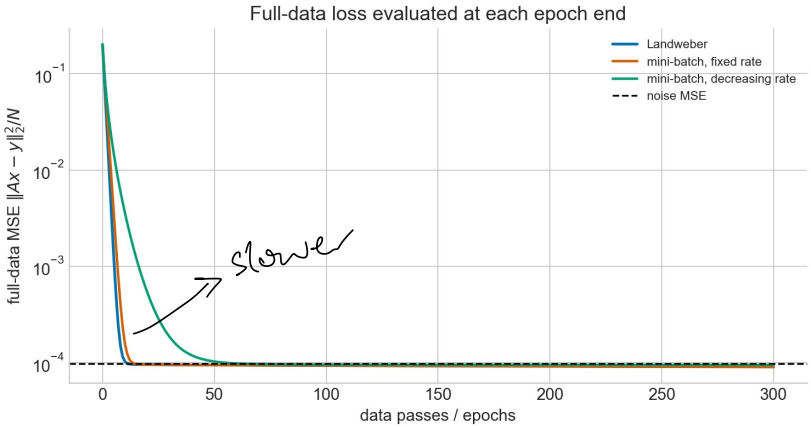
Example: deconvolution again!

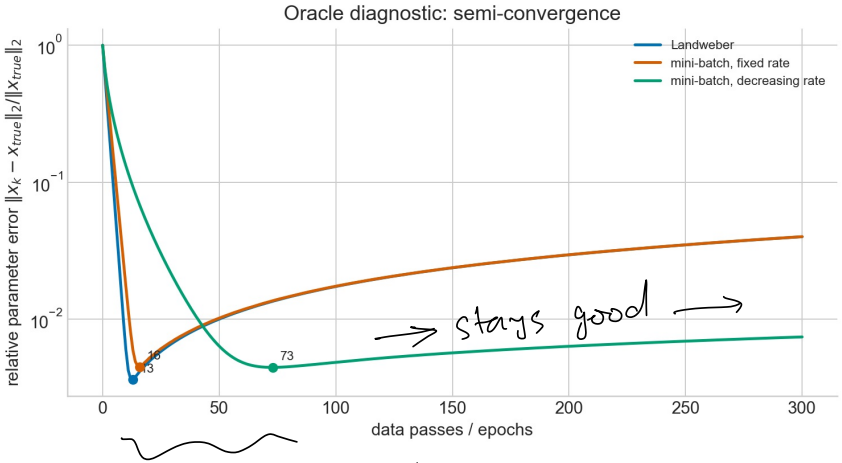
Compare over epochs (seen whole data)



→ decreasing rate: less overfitting

Losses





decreasing rate
↳ not quite as good in best case
↳ much more robust to stepping choice

Exercise:

- Try on polynomial (x^{19}) example!

Tut:

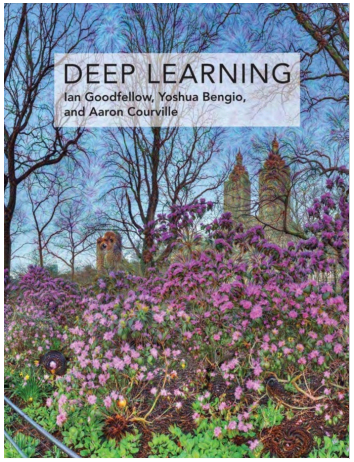
- Describe the minibatch version of gradient descent and contrast it with standard gradient descent. What is the minibatch size for a) stochastic gradient, and b) standard gradient descent. What is another name for standard gradient descent in machine learning terminology.

- Suppose we carry out minibatch gradient descent for three epochs. How many times has the full training data set been used? If we use 20 minibatches, each having 10 samples, how many parameter updates (gradient descent iterations) have occurred after three epochs?

Further info & extensions

References:

(Goodfellow, Bengio & Courville)



Covers extensions eg

learning rates

momentum

general models & losses

further alg.

+ much more!